

Tools

Thursday, March 20, 2025 10:04 PM

Shell Helper

If you are a command line user and want a shortcut to autocomplete commands (and to help with the available arguments), check out a Kafka autocomplete project at <http://mng.bz/K48O>. If you are a Zsh user, you may also want to check out and install Kafka's Zsh-completion plugin from <https://github.com/Dabz/kafka-zsh-completions>.

This is Kafka Jmx exporter and send it to prometheus in prometheus format
https://github.com/danielosi/kafka_exporter

MirroMaker 2.0 : backup tool, backed a cluster with a second cluster between two clusters events are then replicated between topics . There is Confluent Replicator, Cluster Linking are other tools

Kubernetes Confluent Operator API: <https://www.confluent.io/confluent-operator/>
Strimzi: (<https://github.com/strimzi/strimzi-kafka-operator> CNCF project

Apache Flume: Flafka: Flume + Kafka

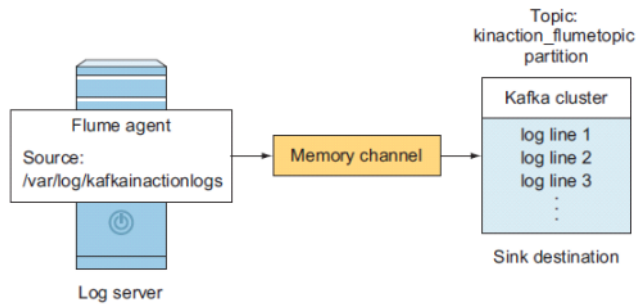


Figure 8.1 Flume agent

have to pull from HDFS directly? Why not just pull it back into Kafka, and our application can process the data as it had before? Loading data into Kafka again is a valid way of reprocessing data that may have aged out of our system. Figure 8.4 shows another example of how we can move data back into Kafka.

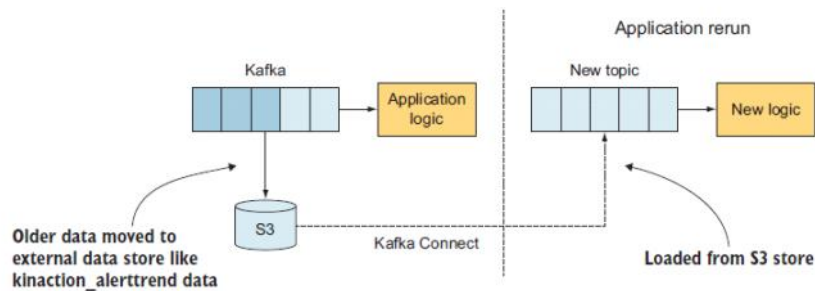


Figure 8.4 Moving data back into Kafka

Confluent - Connect

Apache Flume: to read data and sink to Kafka Cluster
There is also Flafka: kafka and flume integration

Log Rotate:

One tool that you can use for log rotation and compression is (<https://linux.die.net/man/8/logrotate>), but logrotate there are many other tools available as well to manage older server logs. They are not aggregated by default into one location. Various platforms might do this on our behalf, or we can gather them with a tool like Splunk™ (<https://www.splunk.com/>). It is especially important to know when we are trying to analyze logs to gather them when using something like a cloud environment in which the broker instance might not exist.

Prometheus

To extract and store Kafka's metrics data. And send them to Grafana
For the Kafka exporter, an outstanding option is available at https://github.com/danielqsi/kafka_exporter

Grafana

Produce helpful graphical views.

Backups

Kafka does not have a backup strategy like one would use for a database; we don't take a snapshot or disk backup per se. Because Kafka logs exist on disk, why not just copy the entire partition directories? Although nothing is stopping us from doing that, one concern is making a copy of all of the data directories across all locations. Rather than performing manual copies and coordinating across brokers, one preferred option is for a cluster to be backed by a second cluster [23]. Between the two clusters, events are then replicated between topics. One of the earliest tools that you might have seen in production settings is MirrorMaker. A newer version of this tool (called Mirror-Maker 2.0) was released with Kafka version 2.4.0 [24]. In the bin subdirectory of the Kafka install directory, we will find a shell script named kafka-mirror-maker as well as a new MirrorMaker 2.0 script, connect-mirror-maker. There are also some other open source as well as enterprise offerings for mirroring data between clusters. Confluent Replicator (<http://mng.bz/Yw7K>) and Cluster Linking (<http://mng.bz/OQZo>) are also options to be aware of [25].

Strimzi

Another interesting option is Strimzi™ (<https://github.com/strimzi/strimzi-kafka-operator>), if you are looking at running our cluster on Kubernetes. At the time of this writing, Strimzi is a Cloud Native Computing Foundation® (<https://www.cncf.io/>) sandbox project

8.3.1 Apache Flume

If you have ever heard the term Flafka, you have definitely used this Kafka and Flume integration. Flume can provide an easier path for getting data into a cluster and relies more on configuration than on custom code. For example, if you want to ingest data into your Hadoop cluster and already have support from a vendor on these various pieces, Flume is a solid option to get data into your Kafka cluster.

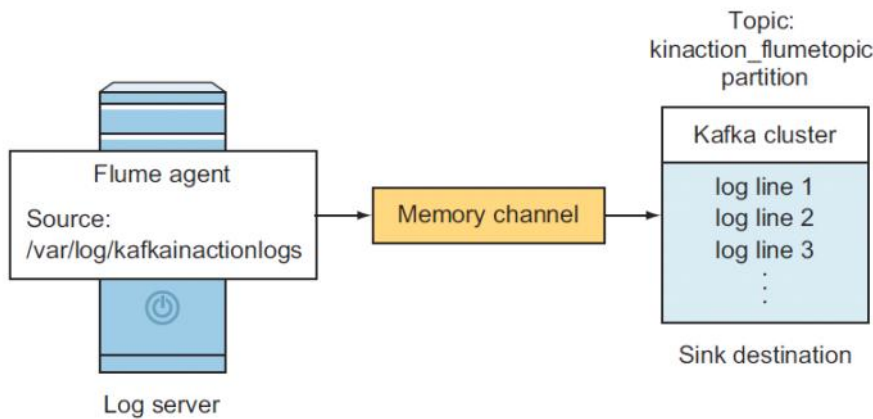


Figure 8.1 Flume agent

to read the file and send the result to a specific Kafka topic.

Listing 8.1 Flume configuration for watching a directory

```

ag.sources = logdir
ag.sinks = kafkasink
ag.channels = c1

#Configure the source directory to watch
ag.sources.logdir.type = spooldir
ag.sources.logdir.spoolDir = /var/log/kafkainactionlogs
...
ag.sinks.kafkasink.channel = c1
ag.sinks.kafkasink.type = org.apache.flume.sink.kafka.KafkaSink
ag.sinks.kafkasink.kafka.topic = kinaction_flumetopic
...
# Bind both the sink and source to the same channel
ag.sources.logdir.channels = c1
ag.sinks.kafkasink.channel = c1
  
```

Defines custom names for the source, sink, and channel

Specific spooldir source lets Flume know which directory to watch for log entries.

This section defines our topic and Kafka cluster information where we want our data to end up.

Attaches the source to the sink by the defined channel

Listing 8.2 Flume Kafka channel configuration

```

ag.channels.channel1.type =
  org.apache.flume.channel.kafka.KafkaChannel
ag.channels.channel1.kafka.bootstrap.servers =
  localhost:9092,localhost:9093,localhost:9094
ag.channels.channel1.kafka.topic = kinaction_channel1_ch
ag.channels.channel1.kafka.consumer.group.id =
  kinaction_flume
  
```

Flume uses the KafkaChannel class as the Kafka channel type.

Provides our servers to connect to

The topic that holds the data between source and sink

Provides a consumer group to avoid collisions with other consumers

Red Hat Debezium:

Debezium (<https://debezium.io>) describes itself as a distributed platform that helps turn databases into event streams. In other words, updates to our database can be treated as events! If you have a database background (or not), you may have heard of the term change data capture (CDC). As the name implies, the data changes can be tracked and used to react to those changes. At the time of writing this chapter, Debezium supports MySQL, MongoDB, PostgreSQL®, Microsoft SQL Server™, Oracle, and IBM Db2. Cassandra™ and Vitess™ are in an incubating status as well [6]. Please see the current list of connectors at <https://debezium.io/documentation/reference/connectors/>. Debezium uses connectors and Kafka Connect to record the events our application consumes from Kafka as a normal client. Figure 8.2 shows an example of Debezium when it is registered as a connector in regard to Kafka Connect.

Secor:

Secor (<https://github.com/pinterest/secor>) is an interesting project from Pinterest that has been around since 2014. It aims to help persist Kafka log data to a variety of storage options, including S3 and Google Cloud Storage™ [7]. The options for

8.4.1 Tiered storage

A newer option from the Confluent Platform version 6.0.0 on is called Tiered Storage. In this model, local storage is still the broker itself, and remote storage is introduced for data that is older (and stored in a remote location) and controlled by time configuration (`confluent.tier.local.hotset.ms`) [8].

Red Hat Debezium

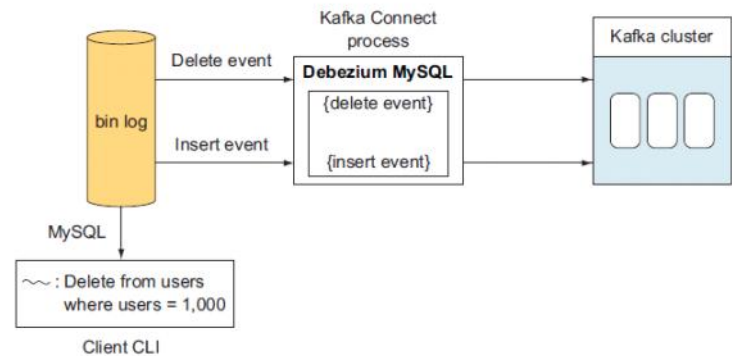


Figure 8.2 Kafka Connect and Debezium used with a MySQL database

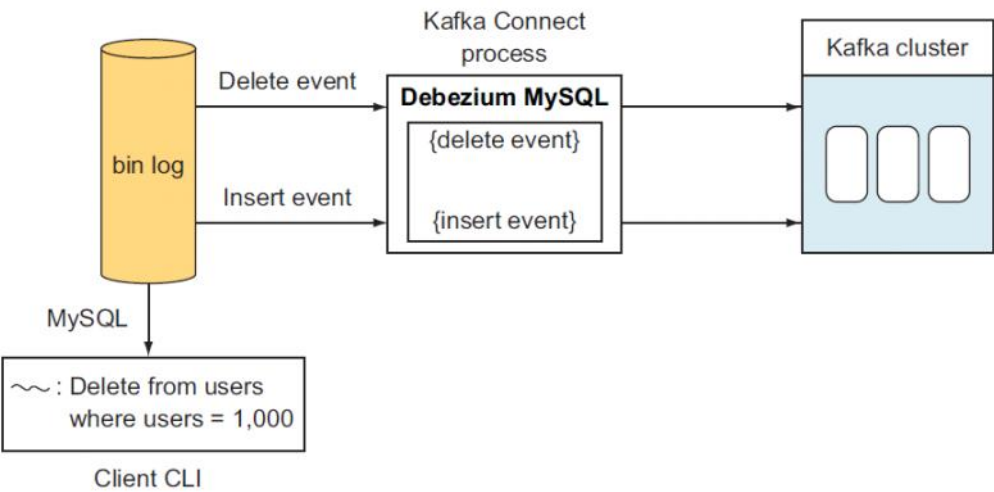


Figure 8.2 Kafka Connect and Debezium used with a MySQL database

In our scenario, a developer uses a command line interface (CLI) and deletes a user against the MySQL database instance that is being monitored for changes. Debezium captures the event written to the database's internal log, and that event goes through the connector service and feeds into Kafka. If a second event, such as a new user, is inserted into the database, a new event is captured

Secor:

Pintereert help persist Kafka Logs to variety of storage options like s3 Gcloud storage. output are also various, including sequence, Apache ORC™, and Apache Parquet™ files as well as other formats. As always, one major benefit of projects having source code in a public repository is that we can see how other teams have implemented requirements that might be similar to ours. Secor runs as a Java process and can be fed our specific configurations. In effect, it acts as another consumer of our existing topic(s) to gather data to end up in a specific destination like an S3 bucket. Secor does not get in the way of our other consumers, and it allows us to have a copy of our events so that they are not lost once Kafka reten Tion removes data from its logs.

Invoking Secor should be familiar to those who are used to working with JARs in a Java environment. We can pass arguments with the standard parameters to the -D Secor application. In this instance, the most important file to update is the properties file with the configuration options. This file lets us fill in the details about our specific cloud storage bucket, for example.

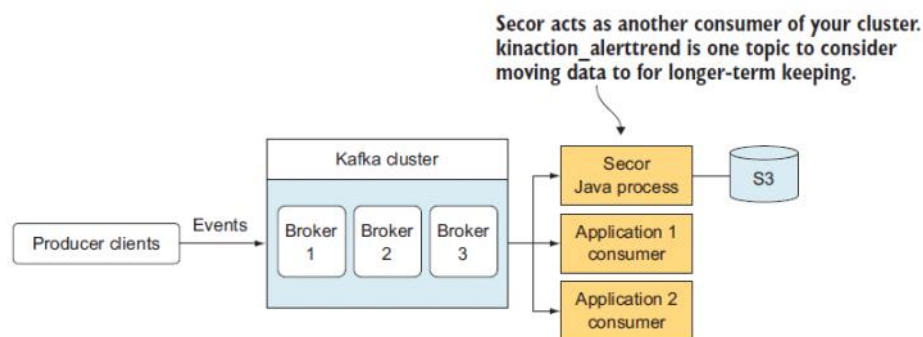


Figure 8.3 Secor acting as a consumer and placing data into storage.

AdminClient

AdminClient is a great tool for building a user-facing application for those who wouldn't normally need or want to use the Kafka shell scripts. It also provides a way to control and monitor what is being done on the cluster.

Kcat:
kcat (<https://github.com/edenhill/kcat>) is a handy tool to have on your workstation, especially when connecting remotely to your clusters

Listing 9.3 Using a kcat producer

```
kcat -P -b localhost:9094 \
  -t kinaction_selfserviceTopic
```

← Sends a broker and topic name from our cluster to write messages to that topic

```
// vs. the shell script we used before
bin/kafka-console-producer.sh --bootstrap-server localhost:9094 \
  --topic kinaction_selfserviceTopic
```

← A reminder of the same functionality as the console producer command

Listing 9.4 Using a kcat consumer

```
kcat -C -b localhost:9094 \
  -t kinaction_selfserviceTopic
```

← Sends a broker and topic name from our cluster to read messages from that topic

```
// vs. the shell script we used before
bin/kafka-console-consumer.sh --bootstrap-server localhost:9094 \
  --topic kinaction_selfserviceTopic
```

← A reminder of the same functionality as the console consumer command

Confluent Rest Proxy Api

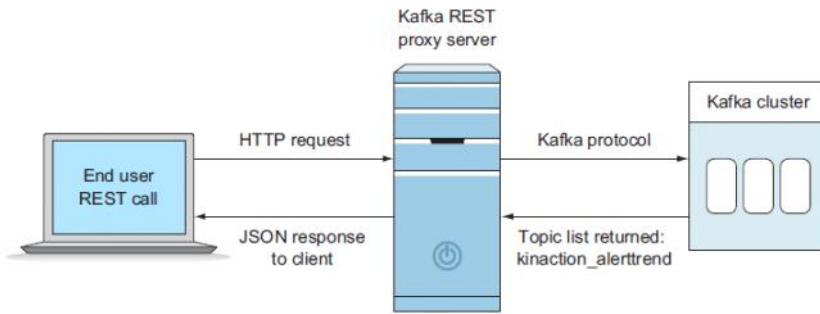


Figure 9.2 The Confluent REST Proxy looks up topics.

Listing 9.5 Starting up a REST Proxy

```
bin/kafka-rest-start.sh \
  etc/kafka-rest/kafka-rest.properties
```

Run this command from the installed Kafka folder to start the REST endpoint.

```
bin/kafka-rest-start.sh \
  etc/kafka-rest/kafka-rest.properties
```

Listing 9.6 A cURL call to the REST Proxy for a topic list

```
curl -X GET \
  -H "Accept: application/vnd.kafka.v2+json" \
  localhost:8082/topics
```

Specifies a format and version

Our target, the endpoint /topics, lists the topics we've created and Kafka's internal topics.

// Output:

```
[ "_confluent.support.metrics", "_confluent-metrics",
  "_schemas", "kinaction_alert" ]
```

Sample output of the curl command

Because this is an evolving API, keep up with the “Confluent REST NOTE Proxy API Reference” at <http://mng.bz/g5Nw> as newer versions come out with more features.

VisualVM (<https://visualvm.github.io/>) is one example. Looking at the available JMX metrics can help us discover points of interest in which we might choose to add

LogRotate : for log rotation

CMAK: Kafka Manager from Yahoo
The Cluster Manager for Apache Kafka, or CMAK (<https://github.com/yahoo/CMAK>), once known as the Kafka Manager, is an interesting project that focuses on managing Kafka as well as being a UI for various administrative activities, and was shared from Yahoo™! One key feature is its ability to manage multiple clusters. Other features include inspection of our overall cluster state and the ability to generate and run partition reassignment. This tool can also deal with authenticating users with LDAP, which might be helpful depending on product requirements for a project [18].

CruiseControl by LinkedIn

Cruise Control (<https://github.com/linkedin/cruise-control>) was created by developers at LinkedIn. Because they have thousands of brokers across their clusters, they have experience running Kafka clusters and have helped codify and automate dealing with some of Kafka's pain points over the years. A REST API can be used as well as the option to use a UI, so we have a couple of ways to interact with this tool. Some of the most interesting features to us are how Cruise Control can watch our cluster and can generate suggestions on rebalancing based on workloads [19].

Confluent Control Center (<https://docs.confluent.io/current/control-center/index.html>) is another web-based tool that can help us monitor and manage our clusters. But one item to note is that it currently is a commercial feature that would need an enterprise license for a production setup. If you already have a subscription to the Confluent platform, there is no reason not to check it out. This tool uses dashboards and can help identify message failures, network latency, and other external connectors